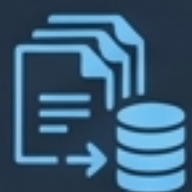
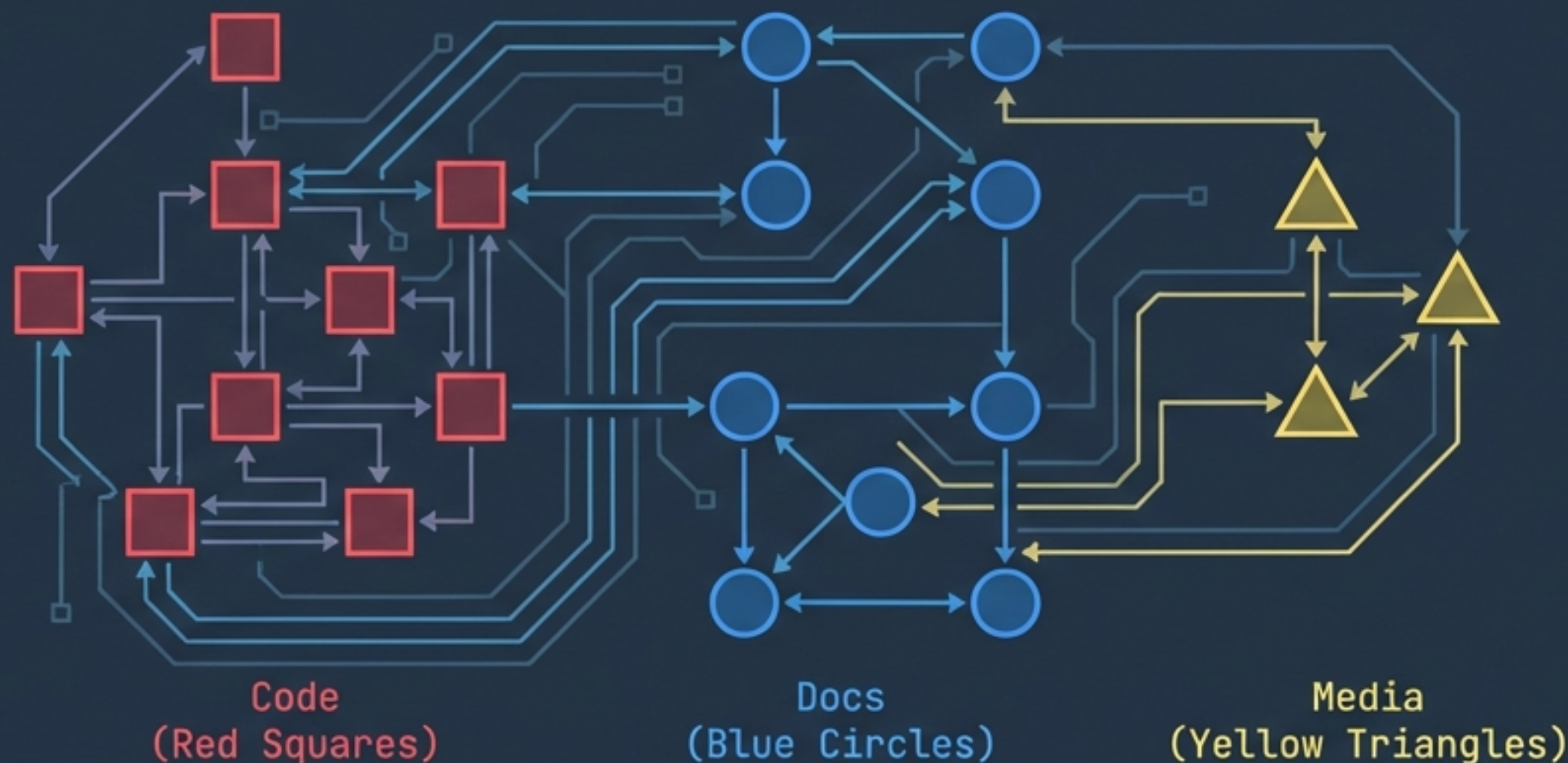


Synthesis: Your AI-Powered Knowledge Infrastructure

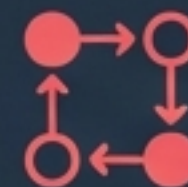
Reveal the hidden structure in your codebase.



Index Everything
(Code, Media, Docs)



Search Intelligently
(Natural Language)



Map Relationships
(Bi-directional)

The Fragmentation of Modern Development

Modern development is messy. Knowledge is scattered, forcing context switching and memory overload.

Source Code



Documentation



Configuration



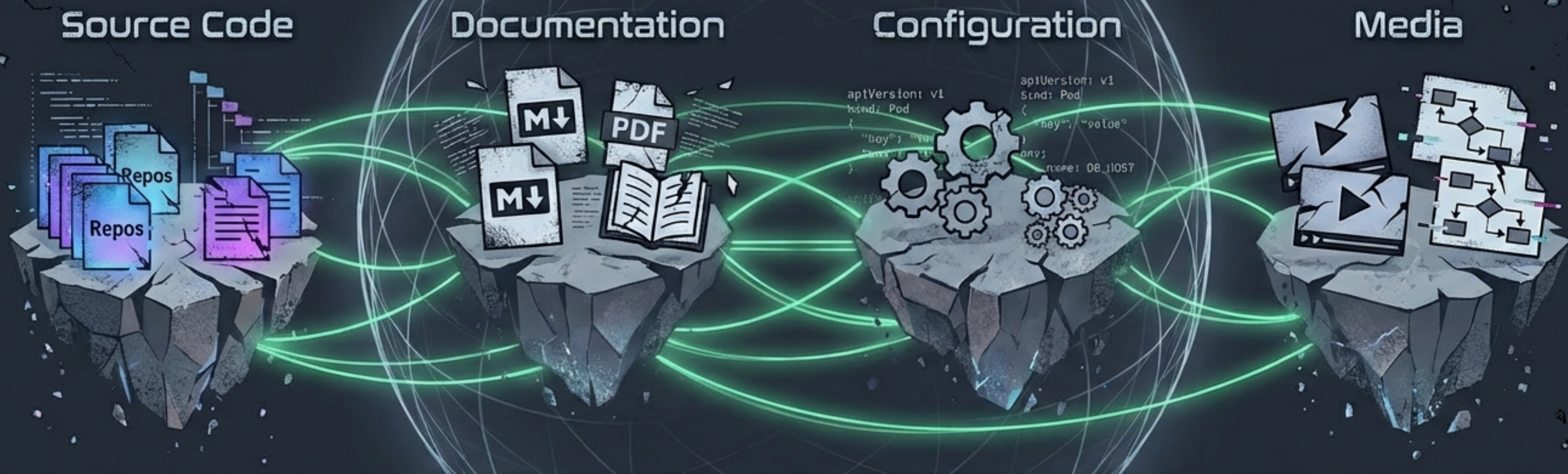
Media



Friction Points

- > Context switching between IDE and Browser
- > Grepping through thousands of files without context
- > Manually tracking dependencies
- > Lost assets: 'Where is that architecture diagram?'

Turn Chaos into a Structured Index



Index Everything

Parses Code, PDFs, Images, and Videos in seconds.

Batteries Included

Video metadata extraction and graph visualization work out-of-the-box.

Local-First & Private

No cloud dependencies.
No API keys required.
Works offline.

Intelligent Search

Lucene-based engine for sub-second queries across thousands of files.

Rapid Onboarding: 5 Minutes to Value

Just a JAR file. Requires Java 17+. No complex server setup.

The Steps

1. Download

```
curl -L ../synthesis.jar -o synthesis.jar
```

2. Initialize

```
java -jar synthesis.jar init
```

3. Index

```
java -jar synthesis.jar scan
```

```
user@dev:~/project$ java -jar synthesis.jar scan  
Synthesis - Scan Workspace
```

```
Phase 1: Scanning directory tree... [Done]
```

```
Phase 2: Analyzing files... [Done]
```

```
-----  
Scan Summary
```

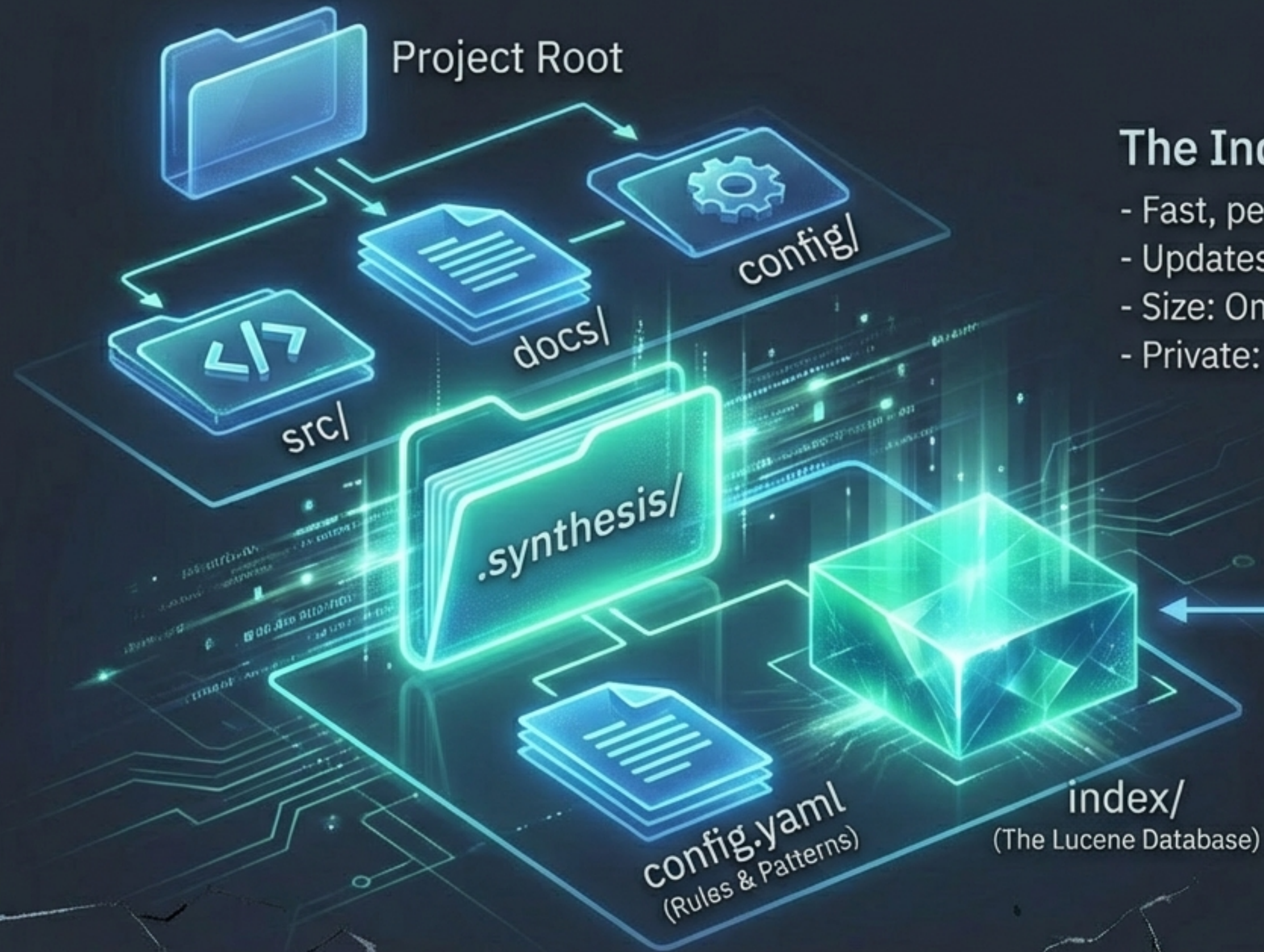
```
Files discovered: 1,987
```

```
Files indexed: 1,987
```

```
Scan duration: 2.3s
```

```
File types: CODE (623), MARKDOWN (234), JSON (89), YAML (41)  
-----
```


Architecture: How Synthesis Works



The Index

- Fast, persistent, and incremental
- Updates via “synthesis scan”
- Size: Only 2-5% of original content
- Private: Lives in your local workspace

Batteries Included Indexing

Synthesis sees what grep misses.



Code Analysis

Language detection (Java, Python, JS, Go, Rust) & Import tracking.



Media Intelligence

Bundled 'ffprobe' automatically extracts duration (12m 34s), resolution (1080p), and format.



Deep Search

Full-text search for PDFs and Markdown link extraction.



Structure Awareness

Parses keys and values in YAML, JSON, and TOML.

Tech Note: FFprobe is bundled and extracted automatically. No external install required.

Command Spotlight: SEARCH

Stop grepping. Start finding.

Terminal

```
user@dev:~$ synthesis search "authentication"
```

1. [CODE] `src/main/java/com/app/AuthService.java`
Java source, 234 lines of code
2. [MD] `docs/security/authentication.md`
Authentication and Authorization Guide
3. [YAML] `config/auth-config.yaml`
YAML configuration with keys: auth, jwt, oauth

Search Features

🔍 **Fuzzy Matching:** Finds "authenticate" from "authentication"

📊 **Intelligent Ranking:**
Code ranked higher for code terms

📄 **Snippet Previews:**
Context-aware display

Command Spotlight: RELATE

Understand impact before you commit.

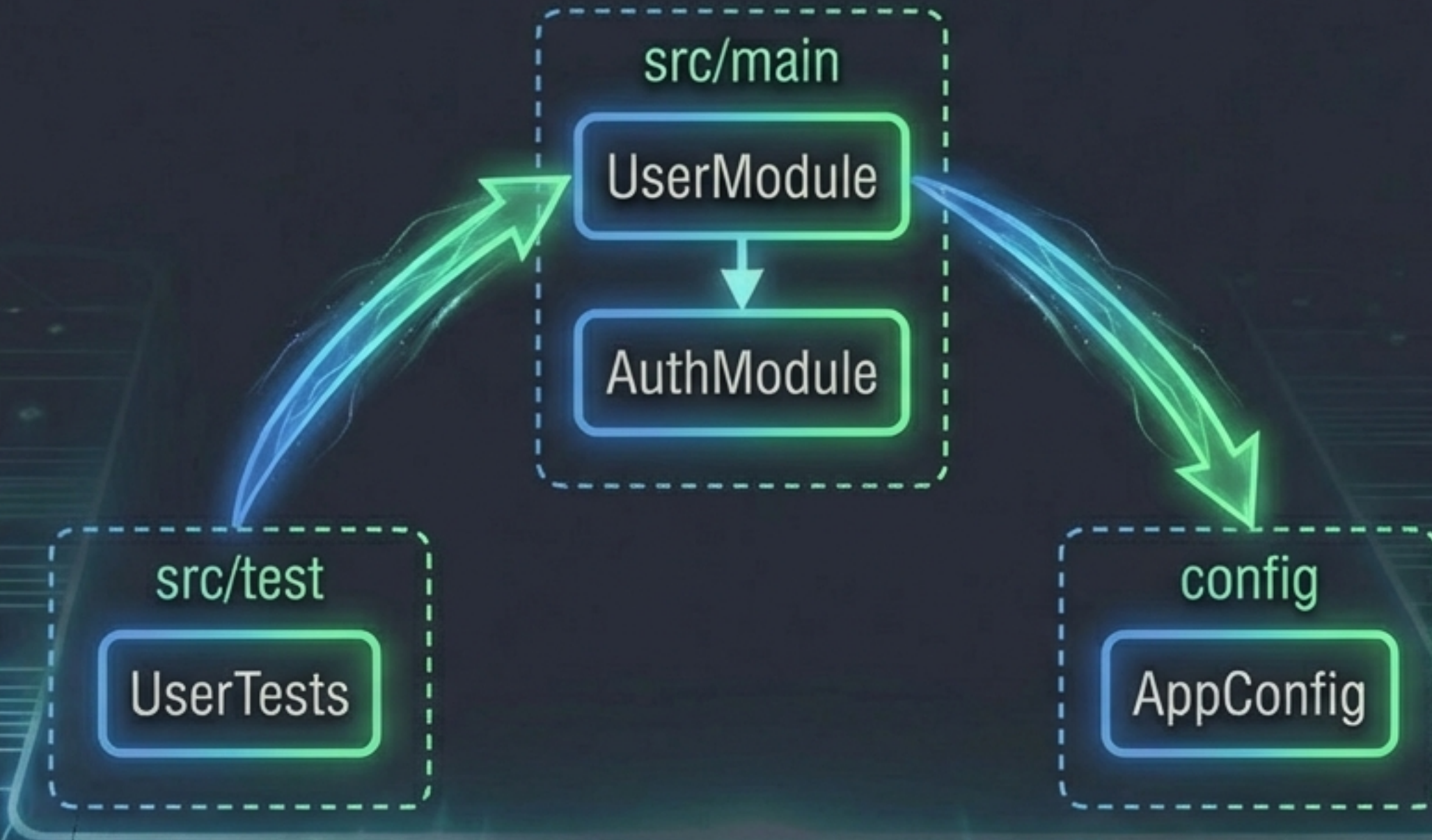


Command: synthesis relate 'AuthService.java'

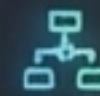
Bi-directional tracking reveals both dependencies and dependents.

Command Spotlight: GRAPH

Turn your codebase into a map.



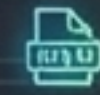
Graph Capabilities



Module Graphs:
High-level overview
(--modules)



Cross-Repo:
Microservices
dependencies (--cross-repo)



Formats: Mermaid,
SVG, PNG, DOT

Command: synthesis graph

Workflow: Rapid Codebase Onboarding

Scenario: A new developer needs to understand a 10,000-file codebase.

1. Scan



``synthesis scan``
(Index knowledge)

2. Overview



``graph --modules``
(Visualize architecture)

3. Discovery



``search 'authentication'``
(Find entry points)

4. Deep Dive



``relate 'AuthService'``
(Trace connections)

Result: Understand architecture in 30 minutes vs. 3 days of reading code.

Workflow: Refactoring & Tech Debt

Scenario: Changing 'UserRepository.java' without breaking the build.

Before



What breaks?

After



`synthesis relate 'UserRepository.java'`

Advanced: Media & Presentations

For when documentation is in video or slide format.

```
$ synthesis search "demo"
```

```
[VID] videos/product-demo.mp4
```

```
Video file: product-demo.mp4 (MP4, 45.2 MB)
```

```
Metadata: [12m 34s] 1920x1080
```

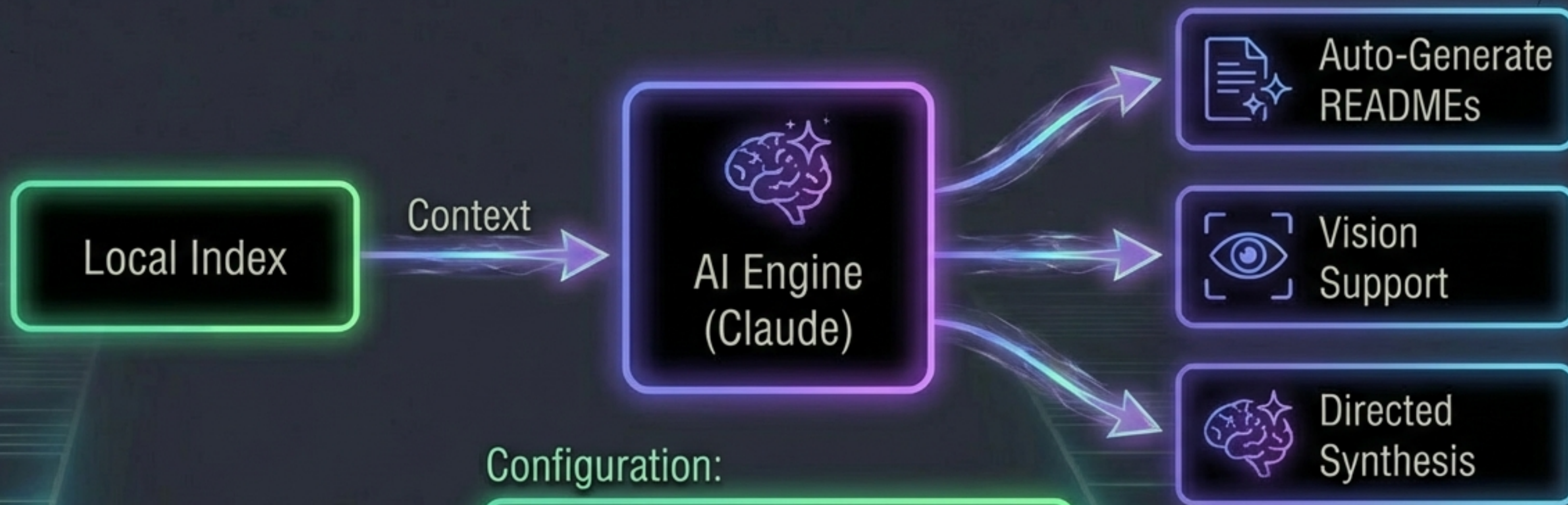
```
[PDF] presentations/demo-slides.pdf
```

```
PDF presentation: Product Demo Slides (24 pages)
```

Tech: Uses
'metadata-extractor'
for speed and bundled
'ffprobe' for broad
format support (MKV,
WebM, AVI).

The AI Layer (Optional Power)

Enhance your index with LLMs (Claude).



Configuration:

```
ai:  
  enabled: true  
  model: "claude-sonnet"  
  readmeGeneration: true
```

Pay-as-you-go. Requires API Key.

Configuration & Pro Tips

config.yaml



```
scan:
  includePatterns:
    - "**/*.py"
  excludePatterns:
    - "**/node_modules"
  maxFileSizeBytes: 10485760
```

Aliases

```
$ alias syn='java -jar synthesis.jar' \
$ alias syngraph='syn graph --modules' \

$ syn search "config"
```

IDE Integration: Add 'synthesis search' as a Task in VS Code or External Tool in IntelliJ.

Summary & Status

Ready to Synthesize.

Essential Commands	Support & Status
• <code>`init`</code>: Initialize workspace • <code>`scan`</code>: Index files (Incremental) • <code>`search "query"`</code>: Find anything • <code>`relate "file"`</code>: Check dependencies • <code>`graph --modules`</code>: Visualize architecture	Supported Types: CODE, MD, PDF, IMG, VID, YAML, JSON Health Check: Run <code>`synthesis status`</code> to verify index health.

Run ``synthesis status`` to check your workspace health.